CODEGRAPH-UPGRADE

CodeGraph 1.5.0 → 1.6.0 — Upgrade Risk Analysis

Status: read-only analysis. Nothing was installed, upgraded, re-indexed, or written to any agent config while producing this document. The only file created is this one.

Analysis date: 2026-08-26 **Analyst method:** inspection of the installed 1.5.0 engine source + the 1.6.0 tarball fetched to a temp dir with `npm pack` (never installed, temp dir removed afterwards).

Verdict: GO — but as a two-phase upgrade, not `codegraph upgrade` on its own. The agent-config risk (RISK 1) is effectively **zero**. The re-index risk (RISK 2) is **real and confirmed**: a full, destructive rebuild of every project index is required.

---

1. Installed vs latest version

Fact	Value	Evidence
Installed version	**1.5.0**	`codegraph version` → `1.5.0`
Installed package `version` field	**1.5.0**	`<home>/.npm-global/lib/node_modules/@colbymchenry/codegraph/package.json`
Binary on PATH	`<home>/.npm-global/bin/codegraph`	`which codegraph`
Latest published	**1.6.0**	`npm view @colbymchenry/`

Fact	Value	Evidence
		`codegraph version` → `1.6.0`
Update-check cache agrees	`{"latest":"v1.6.0"}`	`<home>/.codegraph/update-check.json`
1.6.0 publish date	2026-08-26T17:11:06Z	`npm view … --json` (`time` map) — published *today*

Recent versions (tail of `npm view @colbymchenry/codegraph versions --json`):

```
"1.1.6", "1.2.0", "1.3.0", "1.3.1", "1.4.0", "1.4.1", "1.5.0", "1.6.0"
```

1.6.0 is a genuine `latest` dist-tag, not a pre-release. Note it was published *the same day* as this analysis — there is no soak time behind it. That is a mild argument for waiting a few days, not a blocker.

Package layout (both versions)

The npm package `@colbymchenry/codegraph` is a thin shim. The real engine ships as a platform-specific optional dependency:

- `@colbymchenry/codegraph` — `npm-shim.js`, `npm-sdk.js`, `dist/` **(type declarations only)**, `README.md`
- `@colbymchenry/codegraph-linux-x64` — the actual JS engine (`lib/dist/`) plus a bundled Node runtime. **292 MB unpacked, 941 files.**

Both versions pin `optionalDependencies` to the exact matching version (`1.5.0` → `1.5.0`, `1.6.0` → `1.6.0`), so upgrading the shim always pulls a matching ~292 MB engine download.

---

2. What changed between 1.5.0 and 1.6.0

There is **no CHANGELOG file in the published tarball** (`files` is limited to `npm-shim.js`, `npm-sdk.js`, `dist`, `README.md`). Release notes come from GitHub: `https://github.com/colbymchenry/codegraph/releases` (tag `v1.6.0`, published 2026-08-26T17:07:30Z).

Highlights, verbatim from the 1.6.0 release notes

- **GitHub Copilot is now supported** — `codegraph install` sets it up in VS Code, the Copilot CLI, and JetBrains IDEs.
- **Set up in one command** — `codegraph install --yes --init`.
- **Better answers for your agent** — `codegraph_explore` no longer repeats code it already showed, always returns files/symbols asked for by name, and spends space on code that answers the question rather than look-alikes, generated files, and type shims.
- **Your graph stays right as you keep coding** — a long-running index no longer drifts from a fresh one; edits to `codegraph.json` (such as `exclude`) apply immediately without a restart.
- **No more silent crashes or hangs** — deeply nested C/C++ files, Swift Vapor projects, and large sync batches that used to kill or stall indexing now finish cleanly.
- **A disk-space leak is fixed** — a force-killed session could leave the write-ahead log to grow without bound (tens of gigabytes reported); the leftover log is now folded back and trimmed automatically the next time the project opens.
- **Works from a workspace or monorepo root** — the MCP server finds your indexed project when launched from a folder above it.
- **More accurate code graphs** for TypeScript, Rust, Erlang, C/C++, and Python.
- Also new: per-project Codex setup, a `deprioritize` setting, the `codegraph context` command.

And, critically, this line from the release notes:

> After upgrading, run `codegraph index` once in each project so your existing graph picks up these fixes — `codegraph status` reminds you when it's needed.

Confirmed without installing

`npm view @colbymchenry/codegraph@1.6.0` gives:

- `dist.tarball` = `https://registry.npmjs.org/@colbymchenry/codegraph/-/codegraph-1.6.0.tgz`
- `description` = unchanged from 1.5.0 ("Local-first code intelligence for AI agents (MCP). Self-contained — bundles its own runtime.")
- unpacked size 760.9 kB, 182 files, shasum `d04bc0c93a86186d3ebcb55c74cc15166a50b46e`

Engine tarball: `@colbymchenry/codegraph-linux-x64@1.6.0`, `dist.unpackedSize` = 292,753,805 bytes, `dist.fileCount` = 941.

---

3. Does 1.6.0 bump `EXTRACTION_VERSION` past 24?

YES. 24 → 25.

This is confirmed **twice**, from two independent artifacts, without installing anything.

Evidence A — the published type declaration in the 1.6.0 shim package (`npm pack @colbymchenry/codegraph@1.6.0` in a temp dir, then `package/dist/extraction/extraction-version.d.ts`):

```
export declare const EXTRACTION_VERSION = 25;
```

Evidence B — the actual runnable engine JS of 1.6.0. Streamed a single file out of the 292 MB platform tarball without installing it:

```
curl -sL https://registry.npmjs.org/@colbymchenry/codegraph-linux-
        x64/-/codegraph-linux-x64-1.6.0.tgz \
  | tar -xzO package/lib/dist/extraction/extraction-version.js

exports.EXTRACTION_VERSION = 25;
```

The installed 1.5.0 engine, for comparison: `…/node_modules/@colbymchenry/codegraph-linux-x64/lib/dist/extraction/extraction-version.js:27`

```
exports.EXTRACTION_VERSION = 24;
```

Current index state of this repo (`codegraph status --json`, read-only):


```
{
  "version": "1.5.0",
  "fileCount": 541,
  "nodeCount": 10286,
  "edgeCount": 39793,
  "dbSizeBytes": 48103424,
  "lastIndexed": "2026-08-26T18:34:43.643Z",
  "index": {
    "builtWithVersion": "1.5.0",
    "builtWithExtractionVersion": 24,
```

```
    "currentExtractionVersion": 24,
    "reindexRecommended": false,
    "state": "complete",
    "pendingRefs": 0
  }
}
```

The moment 1.6.0 is installed, `builtWithExtractionVersion: 24 < currentExtractionVersion: 25` and `reindexRecommended` flips to `true`.

`codegraph sync` will NOT clear the staleness

This is the part that makes RISK 2 unavoidable. From the installed engine, `lib/dist/index.js` (the comment is the authors' own, verbatim):

```
// Stamp the index with the engine that built it, so `codegraph
        status`
// and `codegraph upgrade` can recommend a re-index when the
        running
// engine produces richer extraction than the one on disk. Only on
        a
// real full index — a sync touches a subset, so it must NOT
        advance the
// extraction stamp (the bulk would still be stale). See
        extraction-version.ts.
if (result.success && result.filesIndexed > 0) {
    this.queries.setMetadata('indexed_with_version',
        version_1.CodeGraphPackageVersion);
    this.queries.setMetadata('indexed_with_extraction_version',
        String(extraction_version_1.EXTRACTION_VERSION));
}
```

So the post-upgrade advisory's `codegraph sync` option (`# incremental, fast`) will keep the index *working* but leaves it permanently stamped at 24 and permanently flagged stale. Only a full `codegraph index` re-stamps it.

`codegraph index` is destructive — confirmed

`lib/dist/bin/codegraph.js` around line 719:

```
// `index` is a FULL re-index — identical to a fresh `init`.
        RECREATE the
// database from scratch (discard .codegraph/codegraph.db + its
        WAL) rather
// than opening the old graph and DELETE-ing every row.
const cg = await CodeGraph.recreate(projectPath);
```

`CodeGraph.recreate()` in `lib/dist/index.js:292`:

```
static async recreate(projectRoot) {
    ...
    const dbPath = (0, db_1.getDatabasePath)(resolvedRoot);
    try {
        (0, db_1.removeDatabaseFiles)(dbPath);   // <-- deletes
        codegraph.db + WAL/SHM
    }
    catch (err) {
        ...
        throw new Error(`Could not rebuild the index — the
        database file is in use (${reason}). ` +

        `Stop any running CodeGraph MCP server/daemon for this
        project and retry, ...`);
    }
    const db = db_1.DatabaseConnection.initialize(dbPath);   //
        fresh empty DB
    ...
}
```

Practical consequence: `<ext-volume-host>/Projects/vasic/.codegraph/codegraph.db` (46 MB, 541 files, 10,286 nodes, 39,793 edges) is deleted and rebuilt from zero. There is no in-place migration path, and no partial/resumable rebuild. **Back the DB up first** — see §6.

Note also the `EBUSY` failure mode above: any running CodeGraph MCP server/daemon holding the DB can make `recreate()` fail. On Linux (POSIX unlink) this usually succeeds anyway, but a running daemon will then be pointing at a deleted inode. Stop agents first.

---

4. What `codegraph install --refresh` actually writes

It MERGES. It does not overwrite. — with receipts

4a. The refresh is scoped to already-configured targets only

`lib/dist/installer/index.js:340-376`:

```
/**
 * Pure refresh sweep — re-runs `install()` for every target that
        is
 * ALREADY configured at `location` ...
 *
 * Strictly a refresh, never a first install:
 *   - targets that aren't `alreadyConfigured` are skipped
        untouched;
```

```
 *   - permissions are not written (`autoAllow: false`) and the
       prompt
 *     hook is left as-is (`promptHook: undefined`), so choices
       the user
 *     made at install time — or by hand since — are preserved.
 */
function refreshTargets(targets, location) {
    return targets.map((target) => {
        const base = { id: target.id, displayName:
        target.displayName, location };
        if (!target.supportsLocation(location)) {
            return { ...base, status: 'unsupported',
        changedPaths: [] };
        }
        if (!target.detect(location).alreadyConfigured) {
            return { ...base, status: 'not-configured',
        changedPaths: [] };
        }
        const result = target.install(location, { autoAllow:
        false, promptHook: undefined });
        ...
    });
}
```

Two consequences that matter here: the new Copilot targets in 1.6.0 will **not** be auto-added by a refresh (they aren't `alreadyConfigured`), and `~/.claude/settings.json` permissions are **not** rewritten (`autoAllow: false`).

4b. Only 8 targets exist in 1.5.0 — and neither Kimi nor Qwen is one of them

`lib/dist/installer/targets/registry.js`:

```
exports.ALL_TARGETS = Object.freeze([
    claude_1.claudeTarget,      cursor_1.cursorTarget,
    codex_1.codexTarget,        opencode_1.opencodeTarget,
    hermes_1.hermesTarget,      gemini_1.geminiTarget,
    antigravity_1.antigravityTarget, kiro_1.kiroTarget,
]);
```

1.6.0 adds exactly three more — `copilotVscodeTarget`, `copilotCliTarget`, `copilotJetbrainsTarget` — and nothing else.

Grepping every target for `kimi` and `qwen` returns **zero hits in both versions**. The full set of paths any target writes:

Target	Global paths written
claude	

Target	Global paths written
	`~/.claude.json`, `~/.claude/settings.json`, `~/.claude/CLAUDE.md`
cursor	`~/.cursor/mcp.json`
codex	`~/.codex/…`
opencode	`~/.config/opencode/opencode.jsonc` *or* `opencode.json`, `~/.config/opencode/AGENTS.md`
hermes	`~/.hermes/…`
gemini	`~/.gemini/…`
antigravity	`~/.gemini/config`, `~/.gemini/antigravity`
kiro	`~/.kiro/…`

4c. Risk assessment per at-risk file

File	Touched by refresh?	Why
`~/.kimi-code/mcp.json`	**NO — never**	No `kimi` target exists in 1.5.0 or 1.6.0. CodeGraph has no code path that resolves this file.
`~/.qwen/settings.json`	**NO — never**	No `qwen` target exists. The `gemini` target writes `~/.gemini`, not `~/.qwen`.
`~/.claude.json`	**NO — no write will occur**	Merge-by-key writer, and the existing entry is already byte-equal to what it would write → `unchanged`, file untouched. See 4d.
`~/.config/opencode/opencode.json`	**NO — no write will occur**	Surgical JSONC edit, and the existing entry is already byte-equal → `unchanged`,

File	Touched by refresh?	Why
		file untouched. See 4e.

The `lumen` entries in `~/.kimi-code/mcp.json`, `~/.qwen/settings.json`, and `~/.config/opencode/opencode.json` are therefore all safe.

> **Correction to the stated premise:** `~/.claude.json` does **not** currently contain a `lumen` MCP entry. Its `mcpServers` map holds exactly one key: `codegraph`. Lumen reaches Claude Code through a **plugin** (`mcp__plugin_lumen_lumen__*` tools, `enabledPlugins` in `~/.claude/settings.json`), not through `mcpServers`. Nothing in this upgrade touches `enabledPlugins` — `refreshTargets` passes `autoAllow: false`, so `~/.claude/settings.json` is not written at all.

4d. The Claude writer — read-modify-write on one key

`lib/dist/installer/targets/claude.js`:

```
function writeMcpEntry(loc) {
    const file = mcpJsonPath(loc);
    const existing = (0, shared_1.readJsonFile)
        (file);          // <-- reads and PARSES the whole
        existing file
    const before = existing.mcpServers?.codegraph;
    const after = (0, shared_1.getMcpServerConfig)();
    if ((0, shared_1.jsonDeepEqual)(before, after)) {
        // Already exactly what we'd write — preserve byte-
        identical file.
        return { path: file, action: 'unchanged' };            //
        <-- NO WRITE AT ALL
    }
    const action = before ? 'updated' : (fs.existsSync(file) ?
        'updated' : 'created');
    if (!existing.mcpServers)
        existing.mcpServers = {};
    existing.mcpServers.codegraph = after;                     //
        <-- sets ONE key, keeps everything else
    (0, shared_1.writeJsonFile)(file, existing);
    return { path: file, action };
}
```

Note the authors' own comment nearby, which addresses this exact concern:

```
// A pre-existing MCP JSON file (`~/.claude.json` globally,
// `./.mcp.json` locally) containing other MCP servers (no
```

```
// `codegraph` key) is 'updated', not 'created' — we're adding an
// entry to a file that was already there.
```

`readJsonFile` (in `targets/shared.js`) also guards against the unparseable case rather than truncating:

```
/**
 * Read a JSON file, returning `{}` when missing or unparseable.
 *
 * Unparseable files are backed up to `<path>.backup` BEFORE we
       return
 * `{}` — so an idempotent re-run never silently deletes a user's
 * existing config that happened to break JSON parse temporarily.
 */
```

Idempotency check against the live file. `getMcpServerConfig()` returns `{ type: 'stdio', command: 'codegraph', args: ['serve', '--mcp'] }`. The current `~/.claude.json` holds exactly:

```
{"type":"stdio","command":"codegraph","args":["serve","--mcp"]}
```

→ `jsonDeepEqual` is **true** → `action: 'unchanged'` → **`~/.claude.json` is not written at all.** Its 62 top-level keys and 25 project/session entries (122,681 bytes) are untouched.

Confirmed still true in 1.6.0: `diff` of `targets/shared.js` between the 1.5.0 engine and the 1.6.0 engine reports **IDENTICAL**, so `getMcpServerConfig()` is unchanged and the entry still matches.

Worst case if it *had* differed: `writeJsonFile` does `JSON.stringify(data, null, 2)`, which would reformat the whole file (2-space indent, key order preserved) but preserve every key and value. Reformatting, never data loss.

4e. The opencode writer — surgical JSONC edit

`lib/dist/installer/targets/opencode.js`:

```
function writeMcpEntry(loc) {
    const file = configPath(loc);
    ...
    const config = parseConfig(text);
    const before = config.mcp?.codegraph;
    const after = getOpencodeServerEntry();
    if ((0, shared_1.jsonDeepEqual)(before, after)) {
        return { path: file, action: 'unchanged' };            //
        <-- NO WRITE
    }
    ...
```

```
    // Surgical edit — preserves comments, formatting, and order
        of
    // every key we don't touch.
    const edits = (0, jsonc_parser_1.modify)(text, ['mcp',
        'codegraph'], after, {
        formattingOptions: FORMATTING,
    });
    const updated = (0, jsonc_parser_1.applyEdits)(text, edits);
    (0, shared_1.atomicWriteFileSync)(file, updated);
    return { path: file, action: existed ? 'updated' :
        'created' };
}
```

This is the safest writer of the lot — `jsonc-parser`'s `modify`/`applyEdits` rewrites only the byte range of the `mcp.codegraph` value, leaving all 133 other servers, comments, key order, and whitespace physically untouched.

Idempotency check against the live file. `getOpencodeServerEntry()` returns `{ type: 'local', command: ['codegraph', 'serve', '--mcp'], enabled: true }`. The current `~/.config/opencode/opencode.json` holds exactly:

```
{"type":"local","command":["codegraph","serve","--
        mcp"],"enabled":true}
```

→ `jsonDeepEqual` is **true** → `action: 'unchanged'` → **not written.** The 134 servers under `mcp` (including `lumen`), plus the `skills` and `instructions` keys, are safe.

`configPath()` prefers `opencode.jsonc` and falls back to `opencode.json`. Only `opencode.json` exists here, so it resolves to the correct file — no risk of a stray `.jsonc` being created.

`diff` of `targets/opencode.js` between 1.5.0 and 1.6.0: **IDENTICAL**.

4f. What refresh *would* write, in principle

`claudeTarget.install()` also calls `upsertInstructionsEntry(~/.claude/CLAUDE.md)`, and `opencodeTarget.install()` calls it on `~/.config/opencode/AGENTS.md`. That helper is marker-fenced and content-preserving:

```
if (startIdx !== -1 && endIdx > startIdx) {
    const existingBlock = content.substring(startIdx, endIdx +
        endMarker.length);
    if (existingBlock === body) {
        return 'unchanged';
    }
```

```
    const before = content.substring(0, startIdx);
    const after = content.substring(endIdx + endMarker.length);
    atomicWriteFileSync(filePath, before + body + after);   //
        only the fenced block is replaced
    return 'updated';
}
```

Verified: both files carry `<!-- CODEGRAPH_START -->` / `<!-- CODEGRAPH_END -->` at lines 1–10, and a programmatic comparison confirms each on-disk block is **byte-identical** to `CODEGRAPH_INSTRUCTIONS_BLOCK`. `diff` of `instructions-template.js` between 1.5.0 and 1.6.0 is **IDENTICAL**, so these stay `unchanged` too.

`cleanupLegacyHooks()` strips only pre-0.8 `mark-dirty` / `sync-if-dirty` hooks; a scan of `~/.claude/settings.json` finds none. No-op.

4g. The one genuine 1.6.0 installer diff, and why it doesn't apply

`targets/claude.js` is the only changed writer. The diff is entirely a Windows fix (#1466):

```
const PROMPT_HOOK_COMMAND = process.platform === 'win32'
    ? 'codegraph.cmd prompt-hook'
    : 'codegraph prompt-hook';
const PROMPT_HOOK_FORMS = ['codegraph prompt-hook',
        'codegraph.cmd prompt-hook'];
```

plus an in-place migration loop that rewrites an installer-written hook command to the current platform's spelling. On Linux `PROMPT_HOOK_COMMAND` evaluates to `'codegraph prompt-hook'` — exactly what `~/.claude/settings.json` already contains — so `migrated` stays `false` and the function returns `unchanged`. And in a refresh it is never called at all (`promptHook: undefined`).

`installer/index.js` diffs are cosmetic only: help text, prompt hints, and the new `--init` flag.

Bottom line for RISK 1: on this machine, `codegraph install --refresh` under 1.6.0 will write nothing. Every target reports `unchanged`, and Kimi/Qwen are not targets at all.

---

5. Upgrade path that skips the config rewrite

Yes — one exists, and it is what `codegraph upgrade` runs internally anyway.

`lib/dist/upgrade/index.js` detects the install method as `{ kind: 'npm', scope: 'global' }` (the package lives under the global prefix `<home>/.npm-global`) and then runs:

```
function upgradeNpm(method, versionSpec, deps) {
    const args = method.scope === 'global'
        ? ['install', '-g', `${exports.NPM_PACKAGE}@${versionSpec}
        `]
        : ['install', `${exports.NPM_PACKAGE}@${versionSpec}`];
    deps.log(c.dim(`Running: npm ${args.join(' ')}`));
    ...
}
```

So `codegraph upgrade` ≡ `npm install -g @colbymchenry/codegraph@latest` **plus** three post-steps (`upgrade/index.js:355-382`):

1. `reportResolvedVersion()` — a PATH-shadowing probe. Warns if another `codegraph` earlier on PATH shadows the upgraded one.
2. `selfHealPromptHook()` — wires the Claude `UserPromptSubmit` front-load hook if a configured global Claude install lacks it. **Already present here → no-op.**
3. `selfHealInstalledSurfaces()` — spawns `codegraph install --refresh`. **Analysed in §4 → no-op here.**
4. Prints `reindexAdvisory()` — the "run `codegraph sync` / `codegraph index -f`" reminder.

Three ways to run it

Option	Command	Config rewrite	What you miss
A	`codegraph upgrade`	runs `install --refresh` (no-op here)	nothing
B	`CODEGRAPH_NO_INSTALL_REFRESH=1 codegraph upgrade`	**skipped by kill-switch**	agent-surface self-heal (not needed — templates identical)
C	`npm install -g @colbymchenry/codegraph@latest`	**never runs**	PATH probe, prompt-

Option	Command	Config rewrite	What you miss
			hook self-heal, re-index advisory

Option B's kill switch is a first-class, documented feature of the code:

```
function selfHealInstalledSurfaces(deps) {
    if (process.env.CODEGRAPH_NO_INSTALL_REFRESH === '1')
        return;
    if (!deps.hasCommand('codegraph'))
        return;
    ...
}
```

There is a matching `CODEGRAPH_NO_PROMPT_HOOK=1` for step 2.

Recommendation: Option B. It performs the identical binary upgrade, keeps the useful PATH probe and version reporting, and provably cannot touch a single agent config file. Option C is a fine fallback but is strictly less informative — in particular you lose the PATH-shadowing warning, which is the one post-step that catches a real and easy-to-miss failure mode.

---

6. Recommended safe upgrade procedure

Go/no-go: GO. RISK 1 is retired by code inspection — no config write will occur. RISK 2 is real but is a known, bounded, one-time cost (a 46 MB / 541-file rebuild), it is fully backed up by step 2 below, and it is fully reversible by step R2.

The only judgement call: 1.6.0 was published hours before this analysis. If you want soak time, defer a few days — nothing in 1.5.0 is broken. The strongest reason not to wait is the WAL disk-leak fix, which can cost tens of gigabytes if a session is force-killed.

Phase 0 — pre-flight (read-only)

```
codegraph version
codegraph status --json | head -40          # record fileCount /
        nodeCount / edgeCount
which -a codegraph                          # must show exactly
        ONE path
```

Expect `1.5.0`, `builtWithExtractionVersion: 24`, and a single `<home>/.npm-global/bin/codegraph`. More than one entry means a shadowing install — fix that before upgrading.

Phase 1 — back up everything the upgrade could plausibly touch

```
STAMP=$(date +%Y%m%d-%H%M%S)
BK="$HOME/codegraph-upgrade-backup-$STAMP"
mkdir -p "$BK"

# Agent configs (belt and braces — analysis says none will be
        written)
cp -a ~/.claude.json                        "$BK/claude.json"
cp -a ~/.claude/settings.json               "$BK/claude-
        settings.json"
cp -a ~/.claude/CLAUDE.md                   "$BK/CLAUDE.md"
cp -a ~/.kimi-code/mcp.json                 "$BK/kimi-mcp.json"
cp -a ~/.config/opencode/opencode.json      "$BK/opencode.json"
cp -a ~/.config/opencode/AGENTS.md          "$BK/opencode-
        AGENTS.md"
cp -a ~/.qwen/settings.json                 "$BK/qwen-
        settings.json"

# The expensive artifact: the project index
cp -a <ext-volume-host>/Projects/vasic/.codegraph \
      "$BK/vasic-dot-codegraph"

ls -la "$BK"
du -sh "$BK"
```

Record the backup path. Roughly 46 MB plus a few hundred KB.

Phase 2 — stop anything holding the index

Quit Claude Code, opencode, Kimi, Qwen, and any other agent with a live CodeGraph MCP session. `CodeGraph.recreate()` throws on a locked DB, and a live daemon would otherwise be left pointing at a deleted inode.

Phase 3 — upgrade the binary only, with the config rewrite disabled

```
CODEGRAPH_NO_INSTALL_REFRESH=1 codegraph upgrade
```

This downloads ~292 MB (the platform engine bundle). Then verify:

```
codegraph version                                # expect 1.6.0
```

Phase 4 — confirm no config was touched

```
STAMP=...   # the one from Phase 1
BK="$HOME/codegraph-upgrade-backup-$STAMP"
for f in ~/.claude.json ~/.claude/settings.json ~/.kimi-code/
        mcp.json \
         ~/.config/opencode/opencode.json ~/.qwen/settings.json;
        do
  b="$BK/$(basename "$f")"
  # (adjust names to match the copies made above)
  diff -q "$f" "$b" && echo "UNCHANGED: $f" || echo "*** CHANGED:
        $f ***"
done
```

Every line should read `UNCHANGED`. If any reads `CHANGED`, inspect the diff before proceeding — and note that even a "changed" `~/.claude.json` would be a reformat, not a data loss (§4d).

Also spot-check that `lumen` survived:

```
grep -c '"lumen"' ~/.kimi-code/mcp.json ~/.config/opencode/
        opencode.json ~/.qwen/settings.json
```

Phase 5 — confirm the staleness flag, then rebuild

```
codegraph status --json | grep -A5 '"index"'
```

Expect `builtWithExtractionVersion: 24`, `currentExtractionVersion: 25`, `reindexRecommended: true`. That is the expected, correct state — not a fault.

Then, and only with the Phase 1 backup in hand:

```
cd <ext-volume-host>/Projects/vasic
codegraph index
```

This **deletes and rebuilds** `.codegraph/codegraph.db`. Afterwards:

```
codegraph status --json | head -40
```

Expect `builtWithExtractionVersion: 25`, `reindexRecommended: false`, `state: "complete"`, and a `fileCount` at or near 541. A materially lower file count means the rebuild was incomplete — restore from backup (step R2) and investigate.

Repeat `codegraph index` in each other project you care about. `codegraph status` in any project will tell you whether it still needs it.

Phase 6 — restart agents

Restart Claude Code / opencode / Kimi / Qwen so each reconnects to the 1.6.0 MCP server.

Rollback

R1 — revert the binary to 1.5.0 (the exact command):

```
npm install -g @colbymchenry/codegraph@1.5.0
codegraph version        # expect 1.5.0
```

This is safe and complete: `optionalDependencies` are pinned to the exact version, so npm pulls `@colbymchenry/codegraph-linux-x64@1.5.0` back down with it.

Do **not** use `codegraph upgrade 1.5.0` for a downgrade — `runUpgrade` is written around moving to a target version and re-runs the surface self-heals. Plain `npm install -g` is the clean path.

R2 — restore the index (needed if you already ran `codegraph index` under 1.6.0, since a 25-stamped index read by a 1.5.0 engine is not what 1.5.0 built):

```
STAMP=...   # from Phase 1
cd <ext-volume-host>/Projects/vasic
rm -rf .codegraph
cp -a "$HOME/codegraph-upgrade-backup-$STAMP/vasic-dot-
        codegraph" .codegraph
codegraph status --json | head -20     # expect
        builtWithExtractionVersion 24, 541 files
```

R3 — restore agent configs (should never be needed; included for completeness):

```
BK="$HOME/codegraph-upgrade-backup-$STAMP"
cp -a "$BK/claude.json"            ~/.claude.json
cp -a "$BK/claude-settings.json"   ~/.claude/settings.json
cp -a "$BK/CLAUDE.md"              ~/.claude/CLAUDE.md
```

```
cp -a "$BK/kimi-mcp.json"          ~/.kimi-code/mcp.json
cp -a "$BK/opencode.json"          ~/.config/opencode/
        opencode.json
cp -a "$BK/opencode-AGENTS.md"     ~/.config/opencode/AGENTS.md
cp -a "$BK/qwen-settings.json"     ~/.qwen/settings.json
```

---

7. Implications for “keep extensions up to date automatically”

Auto-upgrading CodeGraph is **safe for agent configs** — the writers are merge-only, idempotent, and scoped to already-configured targets, and they never resolve Kimi or Qwen paths at all.

It is **not** safe to auto-upgrade blind with respect to the index. Any release that bumps `EXTRACTION_VERSION` silently converts every project index into a stale one, and the only remedy is a destructive full rebuild that must be scheduled, not stumbled into.

Suggested policy for an auto-update wizard:

1. Before upgrading, capture `codegraph status --json` → `index.currentExtractionVersion` per project.
2. Fetch the candidate version’s `EXTRACTION_VERSION` cheaply and without installing:

   ```
   npm pack @colbymchenry/codegraph@<ver>   # 267 KB, in a temp
           dir
   tar -xzO -f colbymchenry-codegraph-<ver>.tgz \
       package/dist/extraction/extraction-version.d.ts | grep
           EXTRACTION_VERSION
   ```

   The shim package is tiny and its `.d.ts` carries the constant — no need for the 292 MB engine.
3. If the constant is unchanged → auto-upgrade freely.
4. If it bumped → back up each `.codegraph/` dir, upgrade, then queue `codegraph index` per project as explicit, user-visible work.
5. Always set `CODEGRAPH_NO_INSTALL_REFRESH=1` in the automated path, so an unattended upgrade can never reach an agent config file.

Appendix — questions not answerable without a mutating action

Everything asked was answered non-destructively. For completeness, the two things that genuinely cannot be known without mutating:

- **How long the 1.6.0 rebuild of this repo takes, and the resulting node/edge counts.** Only running `codegraph index` (destructive — deletes the DB) reveals this. The 1.5.0 baseline is 541 files / 10,286 nodes / 39,793 edges / 46 MB; 1.6.0's extraction improvements should raise the node and edge counts.
- **That `install --refresh` under the 1.6.0 binary reports `unchanged` in practice.** Proven by source inspection and byte-comparison of every input, but only running `codegraph install --refresh` (a mutating action, by definition) would demonstrate it empirically. Phase 4's `diff` check verifies the same property after the fact, without needing to trust it in advance.

Appendix — commands used for this analysis

All read-only. No global installs, no upgrades, no config writes, no `index`/`init`/`uninit`/`sync`.

```
codegraph version
codegraph status --json
npm view @colbymchenry/codegraph version
npm view @colbymchenry/codegraph versions --json
npm view @colbymchenry/codegraph@1.6.0 --json
npm view @colbymchenry/codegraph-linux-x64@1.6.0 dist.tarball
        dist.unpackedSize dist.fileCount

# 1.6.0 shim, into a mktemp -d, extracted but never installed:
npm pack @colbymchenry/codegraph@1.6.0

# one file streamed out of the 292 MB engine tarball, never
        installed:
curl -sL <engine-tarball> | tar -xzO package/lib/dist/extraction/
        extraction-version.js
curl -sL <engine-tarball> | tar -xz -C <tmp> --wildcards \
     "package/lib/dist/installer/*" "package/lib/dist/upgrade/*"

curl -sL https://api.github.com/repos/colbymchenry/codegraph/
        releases?per_page=5
```

```
# plus reads/greps/diffs of the installed 1.5.0 engine under
# ~/.npm-global/lib/node_modules/@colbymchenry/codegraph/
```

The temp directory was removed after the analysis.